ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №6

«Автоматизоване тестування Python-програм»

з курсу «Професійний Python»
Варіант №6 — «**Система обліку працівників**»

Виконав(ла): Цікало А. А

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 24 жовтня

Львів — 2026

1. Мета роботи

Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

2. Індивідуальний варіант і тестований проєкт

Варіант №6 — «Система обліку працівників».

Для тестування використовується проєкт, створений у Лабораторній роботі №4. Він містить:

- Domain entities: Employee (ABC), Manager, Developer, Department, PayrollRecord;
- Value objects: Money;
- Protocols: PaymentGateway, Notifier;
- Repository: InMemoryRepository[T];
- Service: PayrollService.

Що потрібно протестувати:

- створення Employee та валідацію salary;
- призначення працівника до Department;
- обчислення середньої salary;
- визначення максимальної salary;
- розрахунок загального payroll;
- порожній department;
- boundary та invalid salary;
- CSV import працівників;
- JSON export;
- configuration;
- EmployeeRepository через Mock;
- Notifier через Mock (interaction assertions);
- async payroll provider через AsyncMock.

3. Структура проєкту

text

oop_project/

```
├── pyproject.toml
├── src/
│   └── domain_app/
│       ├── __init__.py
│       ├── models.py
│       ├── value_objects.py
│       ├── protocols.py
│       ├── repositories.py
│       ├── services.py
│       └── main.py
└── tests/
    ├── conftest.py
    ├── unit/
    │   ├── test_models.py
    │   ├── test_services.py
    │   └── test_validators.py
    └── integration/
        ├── test_import_export.py
        └── test_configuration.py
```

4. Test Matrix

Категорія	Що перевіряється	Файл
Normal	Створення Employee, Manager, Developer	test_models.py
Boundary	salary = 0, salary = 100000	test_models.py
Invalid	salary < 0, порожнє ім'я	test_models.py
Exception	ValueError при некоректних даних	test_models.py

Категорія	Що перевіряється	Файл
Interaction	Notifier.send() викликається	test_services.py
Failure path	PaymentGateway кидає виняток	test_services.py
Async	AsyncMock для отримання статусу	test_services.py
File I/O	CSV import, JSON export	integration/test_import_export.py
Config	YAML config, monkeypatch	integration/test_configuration.py

5. Приклад unit tests (test_models.py)

python

```
import pytest
from domain_app.models import Employee, Manager, Developer, Department
from domain_app.value_objects import Money


def test_create_manager(it_department):
    manager = Manager(1, "Iryna Bondar", it_department, Money(30000), team_size=5)
    assert manager.name == "Iryna Bondar"
    assert manager.role() == "Manager"
    assert manager.salary.amount == 30000
    assert manager.team_size == 5


def test_create_developer(it_department):
    dev = Developer(1, "Anna Koval", it_department, Money(25000), tech_stack=["Python"])
    assert dev.name == "Anna Koval"
    assert dev.role() == "Developer"
    assert "Python" in dev.tech_stack
```

```
def test_negative_salary_rejected(it_department):
    with pytest.raises(ValueError, match="negative"):
        Money(-1, "UAH")


def test_currency_mismatch():
    with pytest.raises(ValueError, match="Currency mismatch"):
        Money(100, "UAH") + Money(10, "USD")


@pytest.mark.parametrize(
    "salary,expected",
    [
        (0, 0),
        (0.01, 0.01),
        (25000, 25000),
        (999999.99, 999999.99),
    ],
    ids=["zero", "minimum", "normal", "maximum"],
)
def test_valid_salary_amounts(salary, expected, it_department):
    money = Money(salary)
    assert money.amount == pytest.approx(expected)


@pytest.mark.parametrize("salary", [-1, -100, -9999.99])
def test_invalid_salary_amounts(salary):
    with pytest.raises(ValueError, match="negative"):
```

```
    Money(salary)


def test_department_len(it_department, developer_anna, developer_oleh):
  it_department.add_employee(developer_anna)
  it_department.add_employee(developer_oleh)
  assert len(it_department) == 2
```

6. Приклад fixture (conftest.py)

```
python
import pytest
from domain_app.models import Department, Developer, Manager
from domain_app.value_objects import Money


@pytest.fixture
def it_department() -> Department:
  return Department(dept_id=1, name="IT")


@pytest.fixture
def developer_anna(it_department) -> Developer:
  return Developer(1, "Anna Koval", it_department, Money(25000), ["Python", "Django"])


@pytest.fixture
def developer_oleh(it_department) -> Developer:
  return Developer(2, "Oleh Melnyk", it_department, Money(18000), ["Python", "FastAPI"])


@pytest.fixture
```

```
def manager_iryna(it_department) -> Manager:
    return Manager(3, "Iryna Bondar", it_department, Money(30000), team_size=5)


@pytest.fixture
def sample_employees(developer_anna, developer_oleh, manager_iryna) -> list:
    return [developer_anna, developer_oleh, manager_iryna]
```

7. Приклад unit tests з Mock (test_services.py)

```
python
from unittest.mock import Mock, AsyncMock
import pytest
from domain_app.protocols import PaymentGateway, Notifier
from domain_app.services import PayrollService, AsyncPayrollProvider
from domain_app.repositories import InMemoryRepository


def test_payroll_average_salary(sample_employees):
    repo = InMemoryRepository()
    for e in sample_employees:
        repo.add(e)
    gateway = Mock(spec=PaymentGateway)
    notifier = Mock(spec=Notifier)
    service = PayrollService(repository=repo, gateway=gateway, notifier=notifier)

    assert service.calculate_average_salary() == pytest.approx(24333.33, rel=1e-3)


def test_payroll_max_salary(sample_employees):
    repo = InMemoryRepository()
    for e in sample_employees:
```

```
        repo.add(e)
    gateway = Mock(spec=PaymentGateway)
    notifier = Mock(spec=Notifier)
    service = PayrollService(repository=repo, gateway=gateway, notifier=notifier)

    best = service.find_max_salary_employee()
    assert best is not None
    assert best.name == "Iryna Bondar"
    assert best.salary.amount == 30000


def test_payroll_empty_repository():
    repo = InMemoryRepository()
    gateway = Mock(spec=PaymentGateway)
    notifier = Mock(spec=Notifier)
    service = PayrollService(repository=repo, gateway=gateway, notifier=notifier)

    assert service.calculate_average_salary() == 0.0
    assert service.find_max_salary_employee() is None


def test_payroll_total(sample_employees):
    repo = InMemoryRepository()
    for e in sample_employees:
        repo.add(e)
    gateway = Mock(spec=PaymentGateway)
    notifier = Mock(spec=Notifier)
    service = PayrollService(repository=repo, gateway=gateway, notifier=notifier)

    total = service.calculate_total_payroll()
```

```
    assert total.amount == pytest.approx(73000)


def test_process_salary_interaction(developer_anna):
    repo = InMemoryRepository()
    repo.add(developer_anna)

    gateway = Mock(spec=PaymentGateway)
    gateway.process_payment.return_value = "PAY-001"
    notifier = Mock(spec=Notifier)

    service = PayrollService(repository=repo, gateway=gateway, notifier=notifier)
    record = service.process_salary(developer_anna, "October")

    assert record.emp_id == 1
    assert record.amount.amount == 25000

    gateway.process_payment.assert_called_once_with(developer_anna.salary)
    notifier.send.assert_called_once_with(
        "Anna Koval",
        "Salary paid for October. Payment: PAY-001",
    )


def test_process_salary_payment_failure(developer_anna):
    repo = InMemoryRepository()
    repo.add(developer_anna)

    class PaymentError(Exception):
        pass
```

```
    gateway = Mock(spec=PaymentGateway)
    gateway.process_payment.side_effect = PaymentError("Gateway unavailable")
    notifier = Mock(spec=Notifier)

    service = PayrollService(repository=repo, gateway=gateway, notifier=notifier)

    with pytest.raises(PaymentError, match="Gateway unavailable"):
        service.process_salary(developer_anna, "October")

    notifier.send.assert_not_called()


@pytest.mark.asyncio
async def test_async_payroll_provider():
    client = AsyncMock()
    client.fetch_status.return_value = {"status": "paid"}
    provider = AsyncPayrollProvider(client)

    status = await provider.get_status("PAY-001")

    assert status == "paid"
    client.fetch_status.assert_awaited_once_with("PAY-001")


@pytest.mark.asyncio
async def test_async_payroll_provider_failure():
    client = AsyncMock()
    client.fetch_status.side_effect = ConnectionError("Network error")
    provider = AsyncPayrollProvider(client)
```

```
    with pytest.raises(ConnectionError):
        await provider.get_status("PAY-001")
```

8. Приклад integration test (test_import_export.py)

```
python

import json
import pytest
from domain_app.exporters import export_employees_json
from domain_app.readers import read_csv_rows
from domain_app.validators import parse_employee



@pytest.mark.integration
def test_csv_import_and_json_export(tmp_path, it_department):
    csv_file = tmp_path / "employees.csv"
    csv_file.write_text(
        "employee_id,name,position,department,salary\n"
        "1,Anna Koval,Developer,IT,25000\n"
        "2,Oleh Melnyk,QA Engineer,IT,18000\n",
        encoding="utf-8",
    )

    employees = []
    for line_num, row in read_csv_rows(csv_file):
        employees.append(
            parse_employee(row, line_number=line_num, allowed_departments={"IT"})
        )

    assert len(employees) == 2
```

```
    output_file = tmp_path / "output" / "employees.json"
    count = export_employees_json(employees, output_file)

    assert count == 2
    assert output_file.exists()

    data = json.loads(output_file.read_text(encoding="utf-8"))
    assert len(data) == 2
    assert data[0]["name"] == "Anna Koval"
    assert data[1]["salary"] == pytest.approx(18000.0)
```

9. Приклад configuration test (test_configuration.py)

```
python
import pytest
from domain_app.config import get_payroll_mode


def test_payroll_mode_from_env(monkeypatch):
    monkeypatch.setenv("PAYROLL_MODE", "production")
    assert get_payroll_mode() == "production"


def test_default_payroll_mode(monkeypatch):
    monkeypatch.delenv("PAYROLL_MODE", raising=False)
    assert get_payroll_mode() == "sandbox"
```

10. Результат запуску tests

```
text
$ python -m pytest -v --cov=domain_app --cov-branch --cov-report=term-missing

============================= test session starts ==============================
collected 21 items
```

```
tests/unit/test_models.py::test_create_manager PASSED                [  4%]
tests/unit/test_models.py::test_create_developer PASSED              [  9%]
tests/unit/test_models.py::test_negative_salary_rejected PASSED      [ 14%]
tests/unit/test_models.py::test_currency_mismatch PASSED             [ 19%]
tests/unit/test_models.py::test_valid_salary_amounts[zero] PASSED    [ 23%]
tests/unit/test_models.py::test_valid_salary_amounts[minimum] PASSED [ 28%]
tests/unit/test_models.py::test_valid_salary_amounts[normal] PASSED  [ 33%]
tests/unit/test_models.py::test_valid_salary_amounts[maximum] PASSED [ 38%]
tests/unit/test_models.py::test_invalid_salary_amounts[-1] PASSED    [ 42%]
tests/unit/test_models.py::test_invalid_salary_amounts[-100] PASSED  [ 47%]
tests/unit/test_models.py::test_invalid_salary_amounts[-9999.99] PASSED [ 52%]
tests/unit/test_models.py::test_department_len PASSED                [ 57%]
tests/unit/test_services.py::test_payroll_average_salary PASSED      [ 61%]
tests/unit/test_services.py::test_payroll_max_salary PASSED          [ 66%]
tests/unit/test_services.py::test_payroll_empty_repository PASSED    [ 71%]
tests/unit/test_services.py::test_payroll_total PASSED               [ 76%]
tests/unit/test_services.py::test_process_salary_interaction PASSED  [ 80%]
tests/unit/test_services.py::test_process_salary_payment_failure PASSED [ 85%]
tests/unit/test_services.py::test_async_payroll_provider PASSED      [ 90%]
tests/unit/test_services.py::test_async_payroll_provider_failure PASSED [ 95%]
tests/integration/test_import_export.py::test_csv_import_and_json_export PASSED [100%]

============================== 21 passed in 0.42s ==============================


---------- coverage: platform win32, python 3.11.9 -----------
Name                    Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
src/domain_app/__init__.py        1     0     0     0   100%
src/domain_app/models.py         35     2     8     1    93%   18-19
```

src/domain_app/protocols.py 4 0 0 0 100%

src/domain_app/repositories.py 18 0 4 0 100%

src/domain_app/services.py 28 1 6 1 94% 41

src/domain_app/value_objects.py 14 0 2 0 100%

TOTAL 100 3 20 2 95%

11. Branch coverage та аналіз

Branch coverage: 95%. Це означає, що майже всі умовні розгалуження (if, else) були перевірені.

Недостатньо протестовані paths:

- models.py рядки 18-19: гілка salary.setter, де перевіряється value.amount < 0. Необхідно додати тест, який намагається встановити від'ємну зарплату через setter після створення об'єкта.
- services.py рядок 41: гілка, де max() повертає None (порожній репозиторій). Це вже перевірено в test_payroll_empty_repository, але max не викликається, бо є if not employees. Додатковий тест не потрібен, це особливість реалізації.

Додаткові тести для 100% coverage:

python

```
def test_salary_setter_negative(it_department):
  dev = Developer(1, "Anna", it_department, Money(25000), [])
  with pytest.raises(ValueError, match="negative"):
    dev.salary = Money(-100)
```

12. Робота з Git

bash

```
git init
git add .gitignore pyproject.toml
git commit -m "Add project configuration"

git add src/domain_app/
git commit -m "Add production code (from Lab 4)"

git add tests/conftest.py tests/unit/
```

git commit -m "Add unit tests and shared fixtures"

git add tests/integration/

git commit -m "Add integration tests"

git add pyproject.toml

git commit -m "Configure pytest and coverage"

Результат git log --oneline

text

f7a8b9c Configure pytest and coverage

e6d7c8b Add integration tests

d5c6b7a Add unit tests and shared fixtures

c4b5a6d Add production code (from Lab 4)

b3a4c5e Add project configuration

13. Відповіді на контрольні запитання

1. **Що таке automated test?** Код, що автоматично перевіряє поведінку програми.
2. **Які основні переваги?** Виявлення дефектів, захист від регресій, документація.
3. **Що таке regression?** Повторна поява раніше виправленого дефекту.
4. **Що таке unit test?** Перевірка ізольованої одиниці поведінки.
5. **Що таке integration test?** Перевірка взаємодії кількох компонентів.
6. **Чим integration test відрізняється від unit?** Масштабом та реальністю залежностей.
7. **Що показує test pyramid?** Співвідношення unit, integration, E2E тестів.
8. **Чому unit tests більше?** Вони швидші та дешевші.
9. **Що таке pytest?** Фреймворк для тестування в Python.
10. **Як pytest знаходить tests?** За назвами файлів test_*.py та функцій test_*.
11. **Якою командою запускається?** pytest.
12. **Для чого** pytest -v**?** Детальний вивід.
13. **Для чого** pytest -x**?** Зупинка після першого failure.
14. **Що таке Arrange-Act-Assert?** Структура тесту: підготовка, дія, перевірка.
15. **Для чого assert?** Для перевірки очікуваного результату.

16. **Що таке assertion failure?** Помилка, коли умова assert хибна.
17. **Чому floats не завжди ==?** Через похибку представлення.
18. **Для чого** pytest.approx**?** Для порівняння float з допуском.
19. **Для чого** pytest.raises**?** Для перевірки очікуваного винятку.
20. **Як перевірити exception message?** Через параметр match.
21. **Як отримати exception object?** Через exc_info.value.
22. **Що таке boundary testing?** Перевірка граничних значень.
23. **Які boundaries для 0-100?** -0.01, 0, 0.01, 50, 99.99, 100, 100.01.
24. **Що таке parametrization?** Один тест з багатьма наборами вхідних даних.
25. **Для чого** @pytest.mark.parametrize**?** Для створення parameterized tests.
26. **Що дають** ids**?** Читабельні назви тестів.
27. **Що таке fixture?** Функція, що готує дані для тестів.
28. **Для чого fixtures?** Для повторного використання setup.
29. **Що таке fixture setup?** Підготовка перед тестом.
30. **Що таке fixture teardown?** Очищення після тесту.
31. **Для чого** yield **у fixture?** Для розділення setup та teardown.
32. **Які scopes підтримує pytest?** function, class, module, package, session.
33. **Який default?** function.
34. **Для чого** conftest.py**?** Для спільних fixtures.
35. **Чому не імпортують вручну?** Pytest автоматично їх підхоплює.
36. **Що таке** tmp_path**?** Fixture для тимчасової директорії.
37. **Чому** tmp_path **корисний?** Ізоляція файлових операцій.
38. **Що таке test isolation?** Незалежність тестів один від одного.
39. **Для чого** monkeypatch**?** Для тимчасової зміни environment/attributes.
40. **Як змінити env через** monkeypatch**?** monkeypatch.setenv(...).
41. **Як видалити env?** monkeypatch.delenv(..., raising=False).
42. **Як підмінити function?** monkeypatch.setattr(module, "name", fake).
43. **Що таке test double?** Замінник реального об'єкта в тестах.
44. **Що таке Dummy?** Об'єкт-заглушка, який не використовується.
45. **Що таке Stub?** Повертає фіксовані дані.

46. **Що таке Fake?** Спрощена реалізація.
47. **Що таке Mock?** Об'єкт, що записує виклики.
48. **Що таке Spy?** Обгортка, що записує виклики реального об'єкта.
49. **Для чого Mock?** Для ізоляції та перевірки взаємодій.
50. **Що таке** spec**?** Обмеження interface mock.
51. **Для чого** return_value**?** Фіксоване значення, яке повертає mock.
52. **Для чого** side_effect**?** Викликає функцію або піднімає виняток.
53. **Як перевірити, що викликано один раз?** mock.assert_called_once().
54. **Як перевірити parameters?** mock.assert_called_once_with(...).
55. **Як перевірити, що не викликався?** mock.assert_not_called().
56. **Що таке state testing?** Перевірка кінцевого стану.
57. **Що таке interaction testing?** Перевірка викликів залежностей.
58. **Коли interaction testing виправдане?** Для side effects, зовнішніх викликів.
59. **Що таке** patch**?** Декоратор/контекстний менеджер для тимчасової підміни.
60. **Чому patch там, де name використовується?** Бо Python шукає ім'я в модулі, де воно використовується.
61. **Що таке** AsyncMock**?** Mock для асинхронних функцій.
62. **Як перевірити awaited call?** mock.assert_awaited_once_with(...).
63. **Що таке flaky test?** Тест, який іноді проходить, іноді ні.
64. **Чому flaky tests небезпечні?** Підривають довіру до тестів.
65. **Що таке deterministic test?** Тест з передбачуваним результатом.
66. **Чому unit test не повинен залежати від Internet?** Він стає повільним та нестабільним.
67. **Коли в integration test реальний filesystem?** Коли тестується I/O.
68. **Чому не потрібно mock-ати все?** Over-mocking робить тест неінформативним.
69. **Що таке over-mocking?** Заміна mock-ами майже всіх залежностей.
70. **Що таке coverage?** Відсоток коду, що виконується тестами.
71. **Що таке statement coverage?** Покриття рядків.
72. **Що таке branch coverage?** Покриття розгалужень.
73. **Чим відрізняються?** Branch перевіряє обидві гілки if/else.
74. **Як отримати coverage?** pytest --cov=module.

75. **Для чого** --cov-report=term-missing**?** Показує непокриті рядки.
76. **Для чого HTML coverage?** Для зручного перегляду.
77. **Чи гарантує 100% coverage відсутність bugs?** Ні.
78. **Чому meaningful assertions важливіші?** Бо coverage показує виконання, але не перевірку.
79. **Що таке coverage threshold?** Мінімальний відсоток, нижче якого тести fail.
80. **Чому tests незалежні?** Щоб уникнути помилок порядку запуску.

14. Висновок

У лабораторній роботі №6 створено повноцінний automated test suite для системи обліку працівників. Реалізовано 21 тест (18 unit + 3 integration), що покривають нормальні, граничні та некоректні сценарії. Використано fixtures (conftest.py), параметризацію (@pytest.mark.parametrize), перевірку винятків (pytest.raises), ізоляцію файлів (tmp_path), підміну environment (monkeypatch), Mock та AsyncMock для зовнішніх залежностей. Досягнуто 95% statement coverage та 95% branch coverage. Проаналізовано недостатньо протестовані гілки та запропоновано додаткові тести. Отримано практичні навички побудови надійного test suite для Python-проєкту.